

Programmazione a Memoria Condivisa con OpenMP

Introduzione a OpenMP: filosofia e standardizzazione

All'inizio del calcolo parallelo, scrivere codice per architetture a memoria condivisa richiedeva l'uso di librerie a basso livello (come i pthreads), che comportavano una gestione manuale e complessa della sincronizzazione.

OpenMP (Open specifications for Multi Processing) nasce per risolvere questo problema, offrendo un modello di programmazione parallela a memoria condivisa ad alto livello.

Gestito da consorzio no-profit **OpenMP Architecture Review Board (ARB)** in collaborazione con i principali produttori hardware e sviluppatori software, OpenMP non è un nuovo linguaggio di programmazione. Si tratta, invece, di un'estensione che aggiunge costrutti di programmazione multithreading ai linguaggi C, C++ e Fortran.

Il suo più grande vantaggio architetturale è la possibilità di mantenere un **unico codice sorgente seriale e parallelo**. Il compilatore, tramite direttive specifiche (i pragma), sa quali porzioni di codice parallelizzare; se il supporto a OpenMP non è presente (o viene disabilitato), le direttive vengono semplicemente ignorate, garantendo la perfetta portabilità e l'esecuzione seriale del programma originale.

▼ Evoluzione dello Standard

Nel corso degli anni, OpenMP ha esteso enormemente le sue capacità per adattarsi alle nuove architetture HPC:

- **Fino alla versione 3.1 (07/2011):** focus principale sul classico multithreading per CPU.
- **Versione 4.0 (07/2013):** un aggiornamento rivoluzionario che ha introdotto il supporto per la *thread affinity*, le direttive *SIMD* per la vettorizzazione e, soprattutto, la direttiva *target* per scaricare il calcolo su acceleratori hardware e GPU.
- **Versioni successive (4.5, 5.0, 5.2, fino alla 6.0 prevista per il 11/2024):** continui raffinamenti per l'offloading su dispositivi eterogenei e ottimizzazioni per i moderni supercomputer.

Modello di Esecuzione: Fork / Join

Quando lanci un programma OpenMP, l'esecuzione inizia in modo totalmente seriale con un singolo thread, chiamato **master thread**. Quando il flusso di esecuzione incontra una direttiva che definisce una "regione parallela", OpenMP interviene.

1. **Fork:** il master thread crea (o risveglia da un pool) un team di thread paralleli per dividere il carico di lavoro.
2. **Esecuzione Parallela:** tutti i thread del team eseguono simultaneamente il blocco di codice racchiuso nella regione parallela.
3. **Join:** al termine della regione parallela, è presente una barriera implicita. Tutti i thread attendono che gli altri abbiano finito; dopodiché, i thread "worker" si fermano e solo il master thread prosegue l'esecuzione seriale del programma.

Identificazione e Gestione dei Thread

All'interno di un team, ogni thread riceve un identificativo numerico (TID) che va da 0 (assegnato sempre al master thread) fino a $N - 1$ (dove N è il numero totale di thread attivi). Questo è fondamentale per assegnare manualmente carichi di lavoro specifici.

OpenMP mette a disposizione funzioni di runtime (libreria C) per interrogare lo stato del sistema:

- `omp_get_thread_num()` : restituisce l'ID del thread chiamante.
- `omp_get_num_threads()` : restituisce quanti thread compongono il team nella regione parallela corrente.
- `omp_get_num_procs()` : fornisce il numero di processori fisicamente disponibili nel sistema.

Nota: se queste funzioni vengono invocate all'esterno di una regione parallela (quindi nel codice seriale), restituiranno sistematicamente `tid = 0` e `nt = 1`. Inoltre, l'utilizzo diretto di queste funzioni nel codice lo rende strettamente dipendente da OpenMP, intaccando parzialmente la portabilità del codice seriale se non gestito con macro del preprocessore.

Quanti thread attivare?

Il programmatore ha a disposizione tre livelli gerarchici per definire il numero di thread, con priorità decrescente:

1. **Clausola** `num_threads(N)` : inserita direttamente nella direttiva del pragma (es. `#pragma omp parallel num_threads(8)`), ha la priorità assoluta e sovrascrive qualsiasi altra impostazione per quella specifica regione.
2. **Funzione** `omp_set_num_threads(N)` : imposta a runtime il numero di thread per tutte le regioni parallele future, a meno che non intervenga la clausola vista sopra.
3. **Variabile d'ambiente** `OMP_NUM_THREADS` : impostata dal terminale prima di lanciare un eseguibile (es. `OMP_NUM_THREADS=4 ./a.out`). Viene utilizzata come fallback se non ci sono direttive o chiamate a funzioni specifiche nel codice.

Gestione della Memoria e delle Variabili

Trattandosi di un paradigma a *memoria condivisa*, è vitale comprendere quali thread vedono quali dati. Un errore in questa fase porta a *race condition*.

In OpenMP, le variabili possiedono uno *scope* che può essere principalmente di due tipi:

- **Shared:** tutti i thread leggono e modificano la stessa area di memoria. Per default, tutte le variabili dichiarate *prima e all'esterno* della regione parallela sono condivise.
- **Private:** ogni thread ottiene una copia personale e isolata della variabile. Queste copie sono allocate nello stack (memoria locale) del singolo thread e, per default, non sono inizializzate col valore della variabile originale esterna. Le variabili dichiarate *all'interno* della regione parallela nascono intrinsecamente private.

Direttive Principali e Suddivisione del Lavoro

La sintassi base in C/C++ sfrutta le direttive al preprocessore: `#pragma omp <direttiva> [<elenco clausole>]`. La direttiva padre è `parallel`, che si occupa unicamente di fare Fork dei thread. Tuttavia, per dividere il lavoro reale, dobbiamo annidare (o combinare) altre direttive di *work-sharing*.

Decomposizione di Dominio: la direttiva `for`

Questa è la direttiva più usata nel calcolo scientifico. Divide le iterazioni di un ciclo `for` affinché ogni thread esegua un sottoinsieme di indici lavorando su dati differenti (decomposizione di dominio). Spesso si combina direttamente con la creazione della regione: `#pragma omp parallel for`.

Un concetto chiave è lo **scheduling** (la clausola `schedule`), che determina come i blocchi di iterazioni (chunk) vengono distribuiti:

- **Static:** le iterazioni sono divise equamente e assegnate ai thread a priori prima che il ciclo inizi. È il default (con `chunk = numero_iterazioni / numero_thread`) ed è perfetto se ogni iterazione ha lo stesso identico costo computazionale.
- **Dynamic:** i thread “pescano” iterazioni da una coda condivisa. Il primo thread che si libera dal suo blocco prende in carico il chunk successivo. Introduce un po’ di overhead a runtime, ma è indispensabile per bilanciare il carico (load balancing) se alcune iterazioni richiedono più tempo di altre.

Decomposizione Funzionale: la direttiva `sections`

Se il problema richiede che thread diversi eseguano operazioni *completamente diverse* (decomposizione funzionale), si usa `sections`. All’interno del blocco `#pragma omp sections`, si definiscono blocchi `#pragma omp section` indipendenti. Ogni sezione verrà assegnata ed eseguita da un thread separato.

Controllo di Esecuzione Singola: `single` e `master`

A volte serve che un’operazione (es. I/O o inizializzazione) venga eseguita una volta sola, ma all’interno di una regione parallela.

- `single`: un solo thread (il primo che arriva, in modo non deterministico) esegue il blocco. Gli altri thread saltano l’istruzione ma rimangono in attesa su una **barriera implicita** all’uscita del blocco, per garantire che l’operazione singola sia conclusa prima di proseguire.
- `master`: indica che *esclusivamente* il master thread deve eseguire il blocco. A differenza di `single`, qui **non c’è alcuna barriera implicita**: gli altri thread tireranno dritto senza aspettare il master.

Sincronizzazione

Nei sistemi a memoria condivisa, l’accesso simultaneo ai dati genera conflitti. OpenMP offre strumenti per “mettere in coda” i thread.

- **La barriera** (`barrier`): una sorta di “punto di ritrovo”. Sospende i thread che la raggiungono e la sblocca solo quando *tutti* i membri del team vi sono arrivati, garantendo un punto di sincronizzazione globale.
- **La sezione critica** (`critical`): definisce un blocco di codice che può essere attraversato da **un solo thread alla volta**. Tutti i thread lo eseguiranno, ma in modo mutuamente esclusivo (es. per aggiornare una struttura dati complessa).
- **L’Operazione Atomica** (`atomic`): una versione estremamente ottimizzata (e spesso supportata direttamente dall’hardware) della sezione critica. È limitata esclusivamente all’aggiornamento di una singola posizione di memoria (es. `count++`). Essendo hardware-based, è molto più veloce di `critical`.
- **I Lock di Basso Livello**: se le direttive strutturate non bastano, OpenMP fornisce funzioni in stile pthreads basate sul tipo `omp_lock_t`. Permettono un controllo granulare:
 - inizializzazione (`omp_init_lock`),
 - acquisizione bloccante (`omp_set_lock`),
 - tentativo di acquisizione non bloccante (`omp_test_lock`),
 - rilascio (`omp_unset_lock`),
 - distruzione (`omp_destroy_lock`).

Funzionalità Avanzate per l’HPC Moderno (OpenMP 4.0+)

Dalla versione 4.0 in poi, OpenMP ha smesso di essere un semplice gestore di thread per processori multicore, evolvendosi per le architetture moderne più complesse.

Vettorizzazione (SIMD)

La CPU possiede unità di calcolo vettoriale capaci di processare array di dati con una singola istruzione. La direttiva `#pragma omp simd` forza e “consiglia” al compilatore le strategie migliori per vettorizzare i cicli (spesso suggerendo anche l’allineamento in memoria dei dati, come blocchi da 32 byte) bypassando le normali euristiche conservative dei compilatori C/C++.

Offloading su Acceleratori (Target)

I supercomputer odierni ottengono le loro performance dalle GPU. La direttiva `target` permette a OpenMP di trasferire il controllo del flusso (e la memoria necessaria, tramite clausole di mappatura come `map(tofrom:x)`) dalla CPU host verso un device esterno, eseguendo lì il calcolo intensivo per poi riportare i risultati indietro.

Thread Affinity: Domare le Cache

I moderni server hanno architetture NUMA (Non-Uniform Memory Access) con decine di core fisici. Il sistema operativo tende, per bilanciare il carico, a migrare i thread da un core all’altro durante l’esecuzione. Questa migrazione distrugge la Cache (cache invalidation), poiché il thread spostato non troverà più i suoi dati vicini, causando colli di bottiglia disastrosi sulle prestazioni di calcolo.

L’**Affinity** (Affinità CPU) blocca un thread su un determinato core o risorsa hardware. Dalla versione 4.0, questo è gestibile esplicitamente tramite variabili d’ambiente:

- `OMP_PLACES` : definisce il recinto entro cui il thread può vivere. Può essere il `socket` dell’intero processore (libertà di movimento nel chip), il `core` fisico (libertà tra gli hyper-thread), o un singolo `thread` hardware specifico (nessuna libertà).
- `OMP_PROC_BIND` : definisce la politica di distribuzione dei thread all’avvio. `Close` li posiziona vicini tra loro (utile se condividono molta cache L2/L3), mentre `Spread` li sparpaglia in modo distante (utile per massimizzare l’uso della banda di memoria ed evitare surriscaldamenti locali). Lo stato può essere interrogato a runtime con `omp_get_proc_bind()`.